

```
@NicolasPiquet →/workspaces/esilv-marketpulse-g01-HANETeam (Nicolas_Piquet) $ python src/main.py
```

## PARTIE 1 - Comprendre le programme

1. Quelle fonction lit le JSON ?  
load\_instruments(), avec json.load().
2. Quelle fonction lit le CSV ?  
load\_prices(), avec csv.DictReader().
3. Quelle fonction sélectionne un ticker ?  
filter\_prices(prices, ticker).
4. Quelle fonction coordonne l'exécution ?  
main().
5. Où sont les fichiers d'entrée ?  
Dans data/sample/ : instruments.json et prices.csv.

Rôle des imports et de DATA\_DIR :

- csv : permet de lire le fichier CSV.
- json : permet de lire le fichier JSON.
- Path : permet de construire les chemins des fichiers.
- DATA\_DIR : désigne le dossier contenant les données.

Chemin utilisé : /workspaces/esilv-marketpulse-g01-HANETeam/data/sample

## PARTIE 2 - JSON et dictionnaires

Type de instruments : <class 'dict'>

Clés principales : ['instrument', 'benchmark']

Instrument :

Type : <class 'dict'>

Ticker : AAPL

Nom : Apple Inc.

Devise : USD

Benchmark :

Type : <class 'dict'>

Ticker : SP500

```
@NicolasPiquet →/workspaces/esilv-marketpulse-g01-HANETeam (Nicolas_Piquet) $ python src/main.py
```

Un dictionnaire associe des clés à des valeurs.

instruments contient deux dictionnaires :

instrument et benchmark.

## PARTIE 3 - CSV et conversion numérique

```
Type de prices : <class 'list'>
Type de prices[0] : <class 'dict'>
Colonnes du CSV : ['date', 'ticker', 'open', 'high', 'low', 'close', 'volume']
Première observation : {'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
Close avant conversion : '250.00'
Type avant conversion : <class 'str'>
Close après conversion : 250.0
Type après conversion : <class 'float'>
```

csv.DictReader fournit un dictionnaire par ligne.

list(...) rassemble ces dictionnaires dans une liste.

Par défaut, les valeurs lues dans le CSV sont du texte.

float(...) convertit un cours en nombre décimal.

int(...) convertit un volume entier en nombre entier.

## PARTIE 4 - Filtrer les observations

Nombre total de lignes : 42

AAPL : 21 observations

SP500 : 21 observations

Le filtrage parcourt chaque ligne du CSV.

Si row['ticker'] correspond au ticker recherché,

la ligne est ajoutée à la liste résultat avec append().

Écriture courte équivalente :

```
[row for row in prices if row["ticker"] == ticker]
```

```
@NicolasPiquet →/workspaces/esilv-marketpulse-g01-HANETeam (Nicolas_Piquet) $ python src/main.py
```

```
PARTIE 5 - Premier et dernier cours
```

```
=====
```

```
AAPL :
```

```
    Premier close : 250.00
```

```
    Dernier close : 266.20
```

```
SP500 :
```

```
    Premier close : 6600.00
```

```
    Dernier close : 6742.00
```

```
prices[0] sélectionne la première observation.
```

```
prices[-1] sélectionne la dernière observation.
```

```
['close'] récupère le champ de clôture.
```

```
float(...) convertit ce champ en nombre.
```

```
On suppose les observations déjà classées par date.
```

```
=====
```

```
PARTIES 6 A 8 - Résumé réutilisable
```

```
=====
```

```
=== MarketPulse ===
```

```
Market configuration
```

```
Period   : 1 month
```

```
Interval : Daily
```

```
Instrument
```

```
AAPL - Apple Inc.
```

```
Observations : 21
```

```
First close  : 250.00 USD
```

```
Last close   : 266.20 USD
```

```
Benchmark
```

```
SP500 - S&P 500
```

```
Observations : 21
```

```
First close   : 6600.00
```

```
Last close    : 6742.00
```

```
@NicolasPiquet →/workspaces/esilv-marketpulse-g01-HANETeam (Nicolas_Piquet) $ python src/main.py
```

Pourquoi utiliser une seule fonction d'affichage ?  
Pour réutiliser la même logique avec des données différentes  
et éviter de dupliquer le code.  
show\_currency=False masque la devise pour le benchmark.

Ordre des opérations dans main() :

1. Charger les métadonnées et les prix.
2. Récupérer l'instrument et le benchmark.
3. Filtrer leurs observations.
4. Afficher la configuration et les deux résumés.

```
=====
PARTIE 9 - Concepts Python
=====
```

Liste : collection ordonnée d'éléments.  
Exemple : prices ; prices[0] accède au premier élément.

Dictionnaire : association de clés et de valeurs.  
Exemple : instrument['ticker'].

Fonction : bloc réutilisable défini avec def.  
Exemple : get\_last\_close(prices).

Condition : exécute du code si un test est vrai.  
Exemple : if row['ticker'] == ticker.

Boucle : répète des instructions sur des éléments.  
Exemple : for row in prices.

Conversion : transforme une valeur en un autre type.  
Exemple : float(row['close']).

```
=====
OPTION 1 - Première et dernière date
=====
```

```
@NicolasPiquet →/workspaces/esilv-marketpulse-g01-HANETeam (Nicolas_Piquet) $ python src/main.py
```

```
=====
AAPL :
  Première date : 2026-09-01
  Dernière date : 2026-09-30
SP500 :
  Première date : 2026-09-01
  Dernière date : 2026-09-30
Dates attendues du sample : 2026-09-01 et 2026-09-30.
```

```
=====
OPTION 2 - Volumes strictement positifs
=====
AAPL : 21 observations avec un volume > 0
On convertit le volume avec int() avant la comparaison.
Ce calcul concerne uniquement l'instrument.
```

```
=====
OPTION 3 - Minimum et maximum des clôtures
=====
AAPL :
  Minimum close : 249.80
  Maximum close : 266.20
SP500 :
  Minimum close : 6598.00
  Maximum close : 6742.00
Les cours sont convertis en float avant min() et max().
```

```
=====
OPTION 4 - Les trois premières lignes
=====
prices[:3] sélectionne les trois premières lignes du CSV.
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '3800000'}
{'date': '2026-09-01', 'ticker': 'SP500', 'open': '6592.00', 'high': '6615.00', 'low': '6580.00', 'close': '6600.00', 'volume': '0'}
{'date': '2026-09-02', 'ticker': 'AAPL', 'open': '250.80', 'high': '252.80', 'low': '249.60', 'close': '251.30', 'volume': '3950000'}
```

```
=====
OPTION 5 - Formatage des nombres
=====
Valeur sans formatage : 266.2
Valeur avec .2f       : 266.20
.2f affiche deux chiffres après la virgule.
```